

PRODUCT REQUIREMENTS DOCUMENT

PS017 — Multi-Warehouse Inventory Control & Stock Reconciliation System

OmniStock Retail | Version 1.0 | Development Ready

Product vision: Build a secure, reliable and scalable microservices backend for product catalog management, multi-warehouse inventory, order fulfillment, stock reconciliation and inventory consistency.

1. Product Overview

OmniStock Retail operates multiple regional warehouses and requires a backend platform to track physical stock, manage products, process orders, and maintain accurate inventory during frequent updates.

The platform will use independent microservices for Product, Inventory, Order and Authentication, supported by an API Gateway and Eureka service discovery.

2. Problem Statement

OmniStock needs a centralized system capable of answering: **What products do we have, where are they stored, how much stock is available, and can an order be fulfilled safely?**

- Support multiple warehouses and warehouse-specific stock.
- Maintain accurate inventory during frequent updates.
- Process customer orders safely.
- Prevent overselling and negative inventory.
- Detect low-stock situations.
- Reconcile physical stock against system stock.
- Secure APIs with JWT authentication.
- Support service discovery, gateway routing and load balancing.

3. Product Goals

- Manage the complete product catalog.
- Manage multiple warehouses.
- Track stock for each product in each warehouse.
- Allow authenticated users to place orders.
- Reserve and update inventory safely during order processing.
- Prevent overselling during concurrent requests.
- Provide low-stock detection and replenishment alerts.
- Provide stock reconciliation and audit history.
- Use JWT authentication and role-based authorization.
- Use Eureka service discovery and an API Gateway.
- Support multiple service instances and load balancing.
- Provide automated unit, integration and concurrency tests.

4. Non-Goals

- Frontend/UI.
- Payment gateway integration.
- Shipping-provider integration.
- GPS tracking.
- Full ERP functionality.
- Advanced AI demand forecasting in the initial release.

5. Target Users

Customer

- Authenticate.
- Browse products.
- Place orders.
- View own orders.
- Check order status.

Warehouse Manager

- Monitor inventory.
- View warehouse stock.
- Update stock.
- Reconcile physical stock.
- View low-stock items.
- Review inventory transactions.

Administrator

- Manage products.
- Manage users.
- Perform administrative operations.
- Manage system-level configuration where appropriate.

6. High-Level Architecture

```
CLIENT
|
v
+-----+
| API GATEWAY |
+-----+
|
+-----+-----+
| | |
v v v
AUTH SERVICE PRODUCT SERVICE ORDER SERVICE
|
v
INVENTORY SERVICE
|
v
Product Service

+-----+
| EUREKA SERVER |
| Service Discovery |
+-----+

Auth Service -> Auth DB
Product Service -> Product DB
Inventory Service -> Inventory DB
Order Service -> Order DB
```

7. Microservice Requirements

7.1 Authentication Service

Responsibility: Authentication and authorization.

- User registration
- User login
- BCrypt password hashing
- JWT generation
- JWT validation
- JWT expiration
- Role-based authorization

Suggested roles: ADMIN, MANAGER, CUSTOMER.

```
POST /auth/register
POST /auth/login
```

7.2 Product Service

Responsibility: Product catalog management.

```
Product
-----
id
name
description
price
sku
category
createdAt
updatedAt
```

CRUD APIs

```
POST /products
GET /products
GET /products/{id}
PUT /products/{id}
DELETE /products/{id}
```

Recommended enhancements: pagination, sorting, search, category filtering and unique SKU enforcement.

7.3 Inventory Service

Responsibility: Multi-warehouse stock management.

```
Warehouse
-----
id
name
location
createdAt
updatedAt

Inventory
-----
id
productId
warehouseId
quantity
reservedQuantity
version
updatedAt
```

Available stock should be calculated safely as quantity minus reservedQuantity.

```
POST /inventory
GET /inventory
GET /inventory/product/{productId}
GET /inventory/warehouse/{warehouseId}
GET /inventory/low-stock
POST /inventory/reserve
POST /inventory/{id}/release
POST /inventory/{id}/confirm
PUT /inventory/{id}/reconcile
```

7.4 Order Service

Responsibility: Order creation and coordination with Product and Inventory services.

```
Order
-----
id
customerId
status
totalAmount
createdAt
updatedAt

OrderItem
-----
id
orderId
productId
quantity
price

POST /orders
GET /orders
GET /orders/{id}
GET /orders/customer/{customerId}
PATCH /orders/{id}/status
```

8. Multi-Warehouse Inventory

Stock is maintained independently for each product and warehouse.

Product	Hyderabad	Bangalore	Chennai
Laptop	20	10	5
Mouse	50	30	15
Keyboard	25	20	10

Initial allocation can use a deterministic strategy such as selecting the first warehouse with sufficient available stock. Keep allocation logic isolated so it can later support nearest-warehouse, region-based, priority or multi-warehouse fulfillment.

9. Stock Reservation

Use a reservation/confirmation model rather than immediately treating stock as permanently sold.

```
Order
|
v
Check stock
|
v
Reserve stock
|
v
Create order
```

```
|
+---- Success -> Confirm reservation
|
+---- Failure -> Release reservation
```

If order creation fails after a successful reservation, release the reservation and record the compensating transaction.

10. Low-Stock Management

The system shall identify inventory below a configurable threshold.

```
LOW_STOCK_THRESHOLD = 10

Stock = 25 -> NORMAL
Stock = 8 -> LOW STOCK
```

Expose low-stock inventory with GET /inventory/low-stock. A future or advanced implementation may publish an event or notification to managers.

11. Stock Reconciliation

Warehouse managers must be able to compare system quantity with physically counted quantity.

```
System quantity = 20
Physical quantity = 18
Difference = -2

After reconciliation:
System quantity = 18
```

Every reconciliation must generate an auditable inventory transaction.

```
PUT /inventory/{id}/reconcile

{
  "actualQuantity": 18,
  "reason": "Physical stock count"
}
```

12. Inventory Transaction History

```
InventoryTransaction
-----
id
productId
warehouseId
transactionType
quantity
previousQuantity
newQuantity
reason
referenceId
performedBy
timestamp
```

Suggested transaction types: RESTOCK, SALE, RESERVATION, RELEASE, RECONCILIATION, ADJUSTMENT.

13. Order Placement Workflow

```
Customer
|
v
API Gateway
|
v
JWT Validation
```

```

|
v
Order Service
|
+----> Product Service
| |
| v
| Product exists?
|
+----> Inventory Service
|
v
Check available stock
|
v
Reserve stock
|
v
Create order
|
v
Confirm reservation
|
v
Order CONFIRMED

```

If the product does not exist or stock is insufficient, reject the order. If stock has already been reserved and the order fails, release the reservation.

14. Inter-Service Communication

Use OpenFeign together with Eureka service discovery.

```

@FeignClient(name = "INVENTORY-SERVICE")
public interface InventoryClient {

    @PostMapping("/inventory/reserve")
    InventoryResponse reserveStock(
        @RequestBody ReserveRequest request
    );
}

```

Do not hard-code service URLs in business logic when discovery is available.

15. Eureka Service Discovery

Eureka is the service registry. Services register themselves and clients discover them using logical service names.

```

EUREKA SERVER
|
+-- AUTH-SERVICE
+-- PRODUCT-SERVICE
+-- INVENTORY-SERVICE
+-- ORDER-SERVICE
+-- API-GATEWAY

```

16. API Gateway

The Gateway is the main external entry point.

```

/api/auth/** -> AUTH-SERVICE
/api/products/** -> PRODUCT-SERVICE
/api/inventory/** -> INVENTORY-SERVICE
/api/orders/** -> ORDER-SERVICE

```

- Service discovery

- JWT filtering
- CORS
- Request logging
- Correlation IDs
- Load balancing
- Gateway-level error handling

17. Load Balancing

```
INVENTORY-SERVICE
|
+-- Instance 1 : 8083
|
+-- Instance 2 : 8085

Both instances register as INVENTORY-SERVICE.
```

Requests should be distributed across available instances using service discovery and load balancing.

18. Data Consistency & Concurrency

Inventory consistency is a critical requirement. Inventory quantity must never become negative and concurrent orders must not cause overselling.

```
Stock = 5

Customer A -> orders 4
Customer B -> orders 4

Correct:
A -> SUCCESS, remaining stock = 1
B -> FAILED
```

Use transactional boundaries and appropriate database protection such as `@Transactional`, optimistic locking with `@Version`, pessimistic locking where appropriate, atomic updates and the reservation model.

19. Idempotency

Order creation should optionally support an Idempotency-Key so that repeated requests caused by network retries do not create duplicate orders.

```
Idempotency-Key: <unique-key>
```

20. Security Requirements

- JWT authentication.
- BCrypt password hashing.
- JWT expiration.
- Role-based access control.
- Input validation.
- Secure environment-based secrets.
- No plain-text passwords.
- No committed JWT secrets.
- No sensitive credentials/tokens in logs.

- Proper CORS configuration.

21. Authorization Matrix

Operation	ADMIN	MANAGER	CUSTOMER
View products	✓	✓	✓
Create/update products	✓	Optional	✗
Delete products	✓	✗	✗
View inventory	✓	✓	Optional
Update inventory	✓	✓	✗
Reconcile inventory	✓	✓	✗
View low stock	✓	✓	✗
Place order	✓	Optional	✓
View own orders	✓	✗	✓
View all orders	✓	✓	✗

22. Validation Requirements

- @NotBlank for required text.
- @NotNull for required values.
- @Positive for prices and positive quantities.
- @PositiveOrZero where zero is valid.
- @Size for bounded strings/collections.
- @Email where email addresses are used.

Reject negative quantities, zero order quantities, invalid prices, blank names, duplicate SKUs and invalid user information.

23. Error Handling

Use @RestControllerAdvice and consistent error responses.

```
{
  "timestamp": "2026-09-01T17:00:00",
  "status": 409,
  "error": "INSUFFICIENT_STOCK",
  "message": "Insufficient stock for product LAP-001",
  "path": "/api/orders"
}
```

Recommended status codes: 200 success, 201 created, 400 invalid request, 401 unauthorized, 403 forbidden, 404 not found, 409 conflict, 500 unexpected error, 503 dependent service unavailable.

24. Database Requirements

Use a database-per-service architecture.

```
Product DB:
products

Inventory DB:
warehouses
inventory
```

```
inventory_transactions
```

```
Order DB:  
orders  
order_items
```

```
Auth DB:  
users
```

Services must not directly access or modify another service's database.

25. Performance Requirements

- Use pagination for large collections.
- Index frequently searched fields.
- Avoid N+1 queries.
- Avoid unnecessary database calls.
- Use connection pooling.
- Use transactions appropriately.
- Support multiple service instances.

26. Observability

- Spring Boot Actuator health checks.
- Structured/useful logs.
- Correlation/request IDs.
- Order workflow logging.
- Inventory transaction logging.
- Database/service health information.

Sensitive information such as passwords and JWT secrets must never be logged.

27. Resilience

- Connection and read timeouts.
- Safe error handling for dependent services.
- Circuit breakers/fallbacks where appropriate.
- Retries only when operations are safe to retry.
- Idempotency for operations where repeated requests are possible.

Inventory-changing operations must not be blindly retried because duplicate changes can corrupt stock.

28. API Documentation

Provide OpenAPI/Swagger documentation for endpoints, request/response DTOs, authentication requirements and error responses.

29. Suggested Ports

Service	Port
API Gateway	8080
Auth Service	8081
Product Service	8082
Inventory Service	8083
Order Service	8084
Eureka Server	8761

30. Technology Stack

Component	Technology
Language	Java 17/21
Framework	Spring Boot
Microservices	Spring Cloud
Database	MySQL
ORM	Spring Data JPA / Hibernate
Security	Spring Security + JWT
Discovery	Eureka
Gateway	Spring Cloud Gateway
Communication	OpenFeign
Load Balancing	Spring Cloud LoadBalancer
Testing	JUnit 5, Mockito, Spring Boot Test
API Documentation	OpenAPI/Swagger
Migration	Flyway
Build	Maven
Containerization	Docker

31. Testing Requirements

Unit tests

- Product business logic.
- Inventory allocation.
- Reservation/release.
- Reconciliation.
- Order calculations.
- Authentication and authorization.

Integration tests

- Controller + database.
- Product APIs.
- Inventory APIs.
- Order APIs.

- Authentication APIs.

Microservice tests

- Order → Product.
- Order → Inventory.

Concurrency tests

- Multiple simultaneous orders for limited stock.
- No overselling.
- Reservation consistency.

32. End-to-End Scenario

```
1. POST /api/auth/login
  -> Auth Service
  -> JWT returned

2. POST /api/orders
  -> Gateway validates JWT
  -> Order Service receives request

3. Order Service
  -> Product Service: product exists?

4. Order Service
  -> Inventory Service: reserve quantity

5. Inventory Service
  -> select warehouse with sufficient stock
  -> reserve stock
  -> record transaction

6. Order Service
  -> create CONFIRMED order

7. Inventory Service
  -> confirm reservation

8. Response
  -> order ID + status
```

33. Failure Scenario

```
Initial stock = 2

Customer orders = 5

Inventory:
  -> INSUFFICIENT_STOCK

Order:
  -> FAILED

Inventory:
  -> remains 2
  -> never becomes -3
```

34. Functional Requirements Summary

FR-01: The system shall support JWT authentication.

FR-02: The system shall provide product CRUD and catalog management.

FR-03: The system shall manage multiple warehouses.

FR-04: The system shall track product stock per warehouse.

FR-05: The system shall reserve stock during order processing.

FR-06: The system shall support order creation and retrieval.

FR-07: The system shall reject orders without sufficient stock.

FR-08: The system shall support physical-stock reconciliation.

FR-09: The system shall identify low-stock inventory.

FR-10: The system shall maintain inventory transaction history.

FR-11: Services shall register with Eureka.

FR-12: External requests shall be routed through the API Gateway.

FR-13: The system shall support multiple service instances and load balancing.

FR-14: The system shall prevent negative inventory and overselling.

FR-15: The system shall include automated tests.

35. Non-Functional Requirements

- Security — JWT, BCrypt, RBAC and secure configuration.
- Scalability — services can be independently scaled.
- Reliability — service failures must not silently corrupt inventory.
- Performance — frequent inventory updates should be handled efficiently.
- Maintainability — clear separation of concerns.
- Availability — support multiple service instances.
- Auditability — inventory changes must be traceable.
- Extensibility — warehouse allocation and notifications should be replaceable.

36. Definition of Done

- Eureka Server works.
- All services register with Eureka.
- API Gateway routes requests correctly.
- JWT authentication and role authorization work.
- Product CRUD and search work.
- Warehouse and inventory management work.
- Multi-warehouse stock works.
- Reservation, release and confirmation work.
- Order placement works.
- Insufficient stock is rejected.
- Inventory never becomes negative.
- Concurrent orders cannot oversell.
- Low-stock detection works.

- Stock reconciliation and inventory history work.
- Inter-service communication works.
- Load balancing works.
- Exception handling and validation work.
- Unit, integration and concurrency tests pass.
- API documentation exists.
- Database migrations exist.
- README and deployment instructions exist.

37. Success Criteria

```

LOGIN
|
v
JWT
|
v
API GATEWAY
|
v
ORDER SERVICE
|
+--> PRODUCT SERVICE
|
+--> INVENTORY SERVICE
|
v
Reserve / Update
|
v
Order Confirmed

Must guarantee:
- No negative inventory
- No overselling
- No duplicate orders
- Auditable stock changes
- Secure access
- Service discovery
- Load balancing
- Reliable inter-service communication

```

38. Future Enhancements

- Email/SMS low-stock notifications.
- Kafka/RabbitMQ event-driven workflows.
- Real-time inventory updates.
- Redis caching.
- Distributed tracing.
- Prometheus/Grafana monitoring.
- Advanced warehouse optimization.
- Nearest-warehouse selection.
- Multi-warehouse fulfillment.
- Demand forecasting.

- AI-based replenishment prediction.
- Payment service.
- Shipping service.
- Customer notification service.

39. Final Product Vision

OmniStock should become a reliable, secure and scalable microservices backend that gives OmniStock Retail a single source of truth for products, warehouses, inventory and orders.

The most important business rule is: **An order should only be confirmed when the requested product exists and sufficient inventory can be safely reserved without causing stock inconsistency or overselling.**